

Memtable讲解

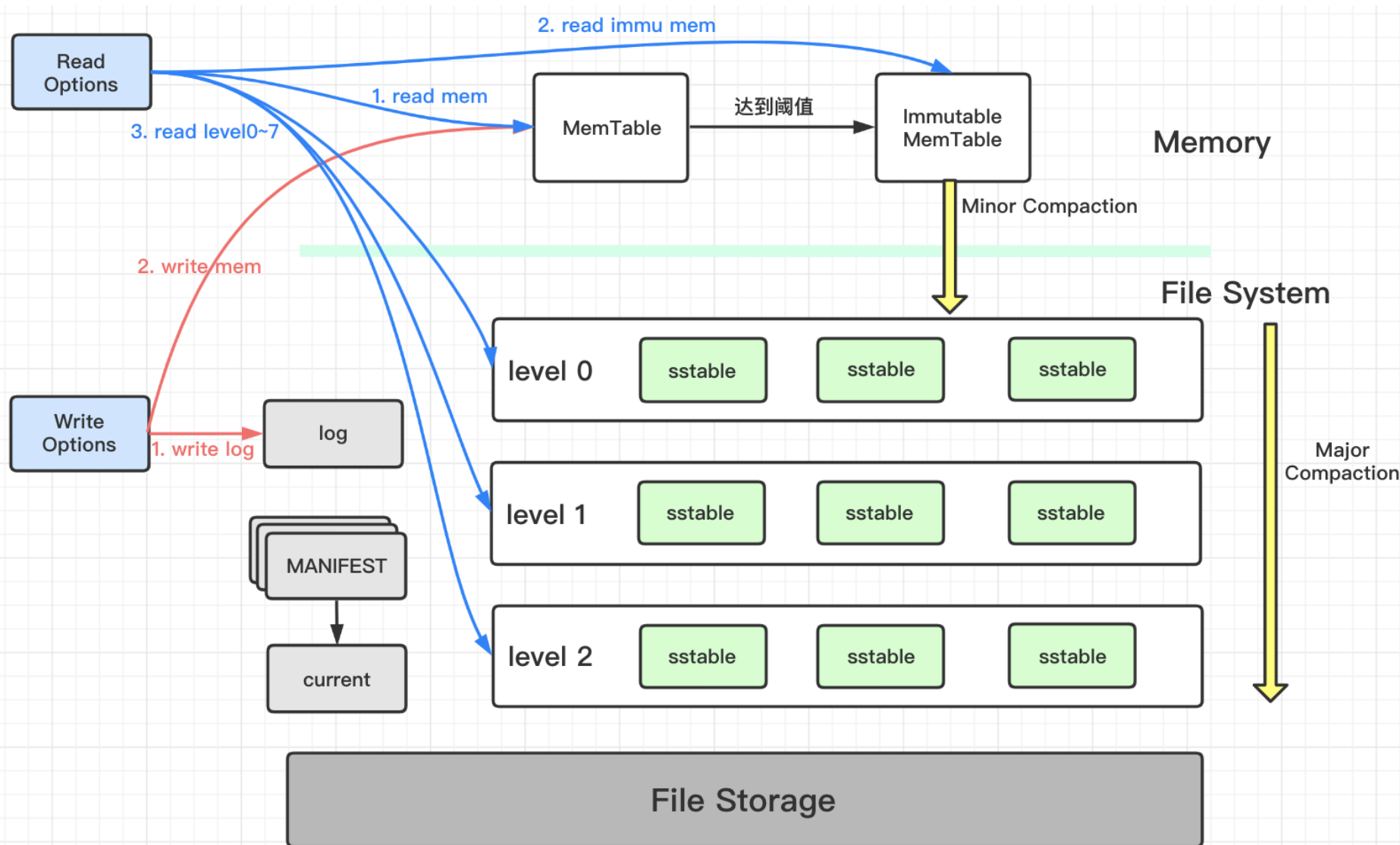
林子骥&郭夏辉

2024.09.26

Topics

1. 为什么要有memtable? 其作用是什么?
2. MakeRoomForWrite函数的作用? 如何把旧Memtable冻结? 写暂停是如何产生的?
3. WriteBatchInternal::InsertInto函数的实现。
4. SkipList类的实现。如何把数据插入SkipList? 如何读取?
5. 如果一个键出现在多个 memtable 中或多个键出现在同一个memtable中, 应向用户返回哪个?
6. 在冻结 memtable 之后, 是否可能有一些线程仍然持有旧的 LSM 状态并写入这些不可变的 memtable? 如何防止这种情况发生?
7. 当不可变的 memtable落盘后还有线程在读这个memtable怎么办?

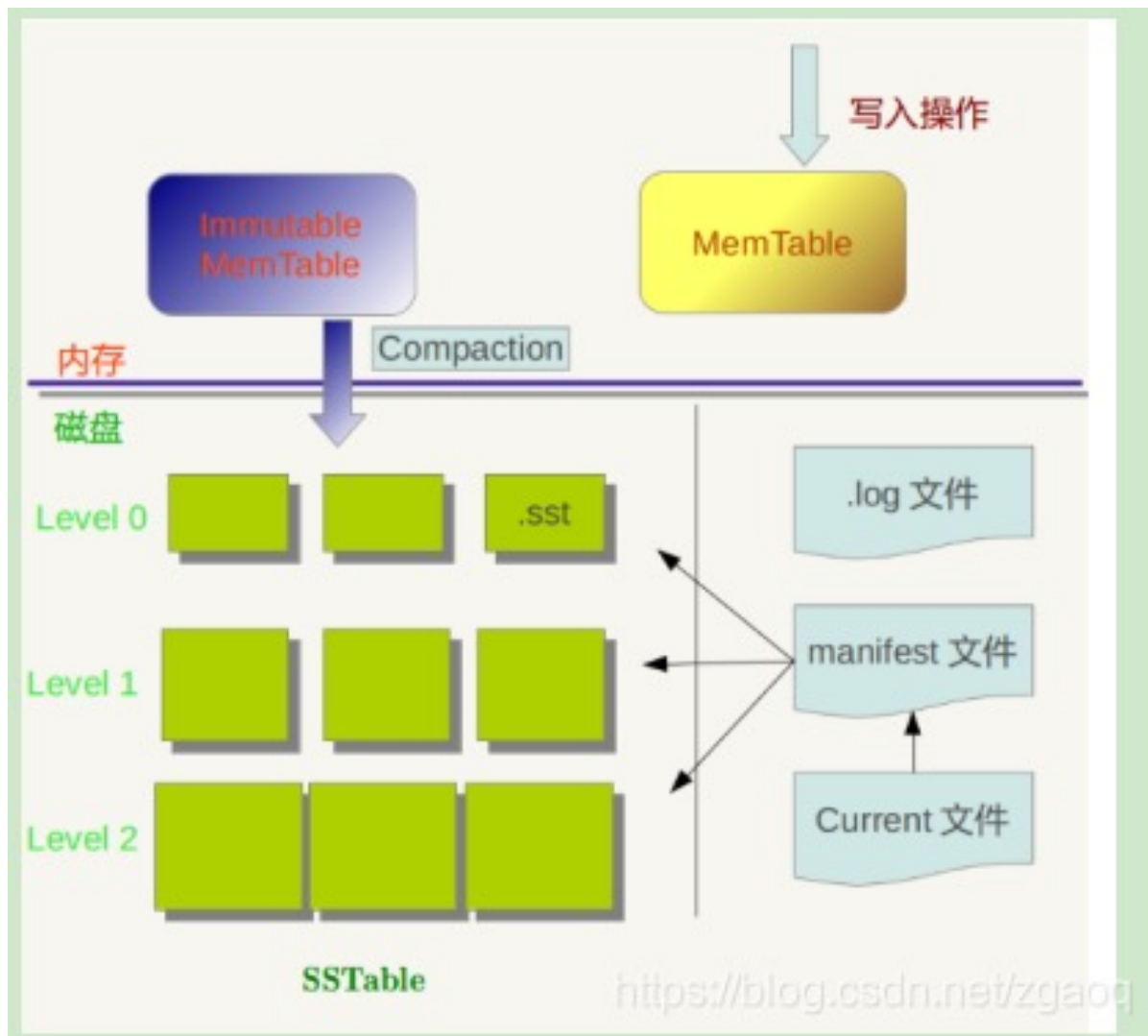
Memtable



- Mem means memory. MemTable是一个内存数据结构，它保存了落盘到SST文件前的数据。
- 是一个内存中的有序map结构，它的底层实现是一个跳表。
- 在进行写入时，总是首先写入 memtable

能看到Memtable同时服务于读和写

Memtable有啥用？



- 加速写操作：减少I/O，增强写入性能
- 批量写入磁盘：积累到一定量后Memtable才被冻结。减少随机写，提升Minor Compaction
- 提供快速查询：新数据在Memtable中，查询很快

项目设计角度，Memtable是内存层面随机写的容器，屏蔽底层操作，方便对arena等进行组合与管理

Immutable Memtable

- 实际上Memtable和Immutable Memtable的结构完全相同
- 只不过前者是可读可写，后者只是可读的而已。

那么为什么要有Immutable Memtable呢？

- 考虑一下没有它的情况。
- memtable满了之后，后台线程需要将memtable**立即**flush到新SSTable中
- 这个过程中难道新的数据还能写入吗？
- 不能！只能在那干等着。新写入的数据处于阻塞状态。
- 但是如果有它，后台线程并不着急flush，不至于出现阻塞。
- 它的作用便是防止写入时的阻塞。

Write: 数据写入的核心方法

DBImpl::Write (db/db_impl.cc)

- 上一Topic的同学已经较好地讲解了Write的工作原理，现在再回顾一下
- 主要流程：
 - 1 每个写线程会将代表的自己的 writer 加入队列
 - 2 当上一个写线程执行完成时会唤醒队列中第一个 writer 线程, 被唤醒的线程会将队列中所有等待写入的数据写进数据库
 - 3 然后将已写入的 writer 的 done 置为 true 并唤醒对应的线程, 被其它线程完成了写入的 writer 会立即返回
- 多线程写->单线程写: 不用考虑memTable的写并发情况了

Write: 数据写入的核心方法

```
MutexLock l(&mutex_); // 将代表自己线程的 writer 加入到队列
writers_.push_back(&w); // 如果自己不是队列中第一个 writer 则通过条件变量 w.cv 等待 w.done 变为 true
while (!w.done && &w != writers_.front()) {
    // 条件变量 CondVar 的 Wait 方法会暂时释放锁，然后阻塞当前线程
    // 在 Wait 期间，其它线程可以拿到锁，并在完成操作后将等待中的线程唤醒
    // 被唤醒的线程将重新拿回锁
    w.cv.Wait();
}
// 能通过 while 进入这里的线程有两种情况：
// 1. 自己就是队列头，需要负责写入
// 2. 自己的数据已被其它线程写入，直接退出就好
// 所有进入此处的线程都持有锁
if (w.done) {
    // 已被其它线程写入，直接返回
    // 返回后 MutexLock 析构，自动释放锁
    return w.status;
}
// May temporarily unlock and wait.
// 循环等待直至数据库状态允许写入
Status status = MakeRoomForWrite(updates == nullptr);
uint64_t last_sequence = versions_>LastSequence();
Writer* last_writer = &w;
```

什么线程能通过阻塞？

1. 自己就是队头（要写入了）
2. 数据已被写入（要退出了）

写入之前要干什么？

数据库允许写了，但是空间不够就尴尬了☹

MakeRoomForWrite职责所在

写入流程

```
{
    //memTable是线程安全的
    //unlock允许其他write放入writers队列中
    // 解锁之后，在当前线程写入期间其它线程就可以将自己的 writer 入队了
    mutex_.Unlock();
    // 第一步：写入 WAL 日志
    status = log_>AddRecord(WriteBatchInternal::Contents(write_batch));
    bool sync_error = false;
    if (status.ok() && options.sync) {
        status = logfile_>Sync();
        if (!status.ok()) {
            sync_error = true;
        }
    }
    // 第二步：写入 MemTable
    if (status.ok()) {
        status = WriteBatchInternal::InsertInto(write_batch, mem_);
    }
    mutex_.Lock(); // 后续将对writers队列操作
    if (sync_error) {
        // The state of the log file is indeterminate: the log record we
        // just added may or may not show up when the DB is re-opened.
        // So we force the DB into a mode where all future writes fail.
        RecordBackgroundError(status);
    }
}
if (write_batch == tmp_batch_) tmp_batch_>Clear();
// 第三步：更新 last_sequence
versions ->SetLastSequence(last_sequence);
```

也是3步：写入 WAL 日志，写入MemTable，更新last_sequence

具体的写入使用的是
InsertInto

可以看到：1.线程写入Memtable的过程中是无锁的；2.日志先写

MakeRoomForWrite

DBImpl:: MakeRoomForWrite(db/db_impl.cc)

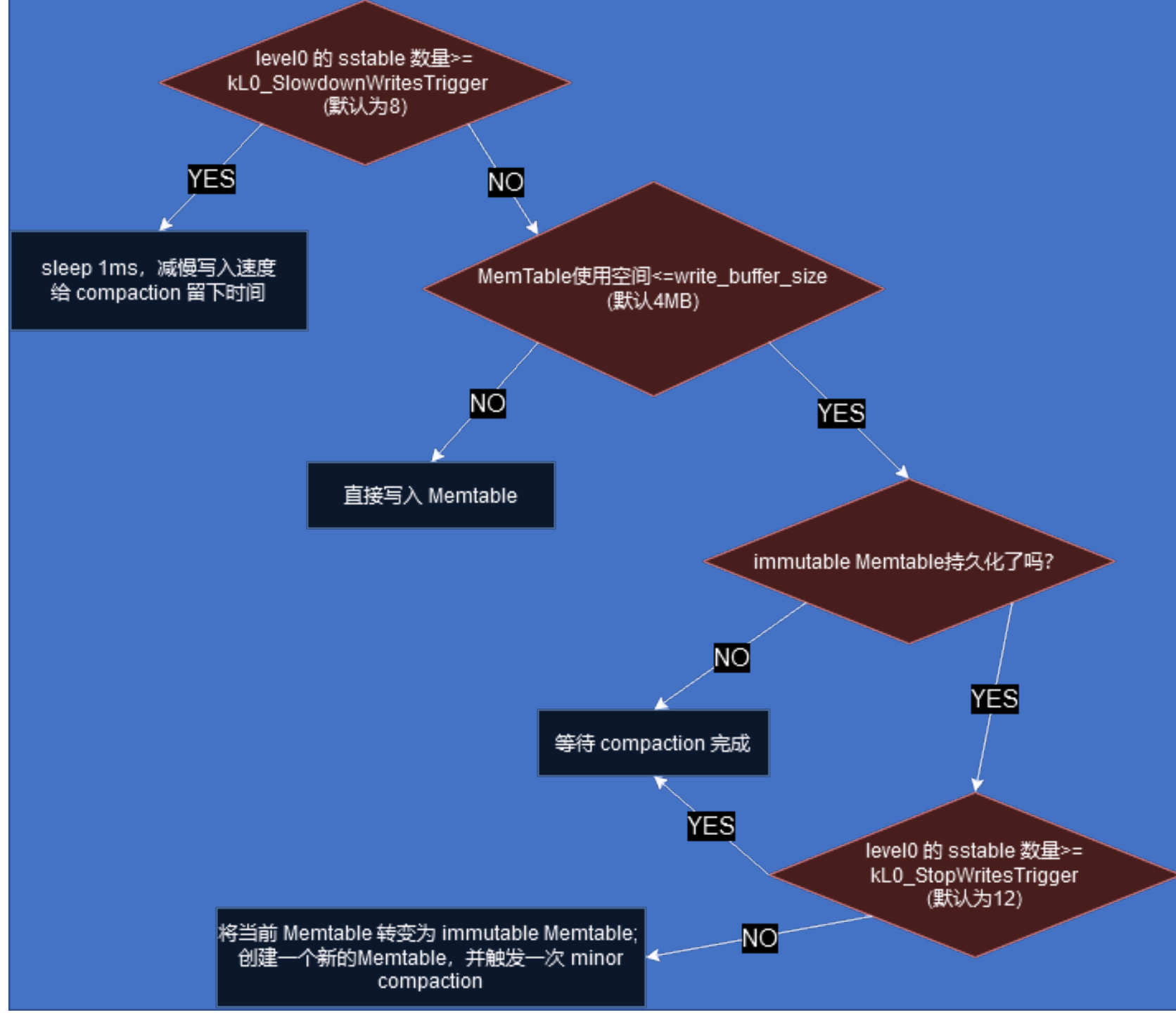
```
Status DBImpl::MakeRoomForWrite(bool force) {  
    mutex_.AssertHeld();  
    assert(!writers_.empty());  
    bool allow_delay = !force;  
    Status s;  
    while (true) {  
        if (!bg_error_.ok()) {  
            // Yield previous error  
            s = bg_error_;  
            break;  
        } else if (allow_delay && versions_->NumLevelFiles(0) >=  
                    config::kL0_SlowdownWritesTrigger) { ...  
        } else if (!force &&  
                    (mem_->ApproximateMemoryUsage() <= options_.write_buffer_size)) { ...  
        } else if (imm_ != nullptr) { ...  
        } else if (versions_->NumLevelFiles(0) >= config::kL0_StopWritesTrigger) { ...  
        } else { ...  
    }  
    return s;  
}
```

- 写入前检查
不适合写入则循环等待
比如说等到后台 compaction
完成

MakeRoomForWrite

代码太长，我们看流程图。

好吧，流程图也有点复杂💡



如何把旧的memtable冻结?

```
} else {  
    // Attempt to switch to a new memtable and trigger compaction of old  
    // 创建一个新的 WAL 日志  
    assert(versions_->PrevLogNumber() == 0);  
    uint64_t new_log_number = versions_->NewFileNumber();  
    WritableFile* lfile = nullptr;  
    s = env_->NewWritableFile(LogFileName(dbname_, new_log_number), &lfile);  
    if (!s.ok()) {  
        // Avoid chewing through file number space in a tight loop.  
        versions_->ReuseFileNumber(new_log_number);  
        break;  
    }  
    delete log_;  
    s = logfile_->Close();  
    if (!s.ok()) { ...  
    delete logfile_;  
  
    logfile_ = lfile;  
    logfile_number_ = new_log_number;  
    log_ = new log::Writer(lfile);  
    // 将当前 Memtable 置为 immutable, 并创建一个新的 mutable MemTable  
    // 由于随后 mem_ 会指向新的 MemTable, 由 mem_ 引用变为了 imm_ 引用, 引用计数不变, 不需要调用 Unref  
    imm_ = mem_;  
    has_imm_.store(true, std::memory_order_release);  
    mem_ = new MemTable(internal_comparator_);  
    mem_->Ref();  
    force = false; // Do not force another compaction if have room  
    MaybeScheduleCompaction();  
}
```

When?

Memtable达到最大值且没有immutable Memtable时

How?

1. 将当前 Memtable 转化为 Immutable Memtable
2. 创建一个新的Memtable
3. 触发一次 Minor compaction

MaybeScheduleCompaction?

old immutable memtable -> L0
包括minor compaction

写暂停如何产生的?

```
} else if (imm_ != nullptr) {  
    // We have filled up the current memtable, but the previous  
    // one is still being compacted, so we wait.  
    Log(options_.info_log, "Current memtable full; waiting...\n");  
    background_work_finished_signal_.Wait();  
} else if (versions_->NumLevelFiles(0) >= config::kL0_StopWritesTrigger) {  
    // There are too many level-0 files.  
    Log(options_.info_log, "Too many L0 files; waiting...\n");  
    background_work_finished_signal_.Wait();  
} else {
```

When?

当前0级文件数量大于等于kL0_StopWritesTrigger
或者Immutable memtable还未持久化

How?

等待compaction完成，直到被唤醒，阻塞线程将wait

Introduction to InsertInto

```
// 第二步: 写入 MemTable
if (status.ok()) {
    status = WriteBatchInternal::InsertInto(write_batch, mem_);
}
```

Writer具体的写入Memtable

InsertInto的代码:

```
Status WriteBatchInternal::InsertInto(const WriteBatch* b, MemTable* memtable) {
    MemTableInserter inserter;
    inserter.sequence_ = WriteBatchInternal::Sequence(b);
    inserter.mem_ = memtable;
    return b->Iterate(&inserter);
}
```

看起来好简单

No! 实际情况比较复杂, 我们一步一步来拆解开细谈。

写入和删除的入口 (db/db_impl.cc)

```
Status DB::Put(const WriteOptions& opt, const Slice& key, const Slice& value) {  
    WriteBatch batch;  
    batch.Put(key, value);  
    return Write(opt, &batch);  
}
```

```
Status DB::Delete(const WriteOptions& opt, const Slice& key) {  
    WriteBatch batch;  
    batch.Delete(key);  
    return Write(opt, &batch);  
}
```

这两个方法都首先被打包成一个WriteBatch，然后再调用Write函数
我们接下来重点关注WriteBatch

WriteBatch的定义

(include/leveldb/write_batch.h)

WriteBatch接口中除了提到的Put和Delete， 还提供了一个Append方法可以将其他WriteBatch合并过来。

另外提供了Iterate迭代函数和对应的Handler类接口， 后面会使用到。

它只有一个私有成员 rep_。

```
class LEVELDB_EXPORT WriteBatch {
public:
    class LEVELDB_EXPORT Handler { ...
    WriteBatch();
    // Intentionally copyable.
    WriteBatch(const WriteBatch&) = default;
    WriteBatch& operator=(const WriteBatch&) = default;
    ~WriteBatch();
    // Store the mapping "key->value" in the database.
    void Put(const Slice& key, const Slice& value);
    // If the database contains a mapping for "key", erase it. Else do nothing.
    void Delete(const Slice& key);
    // Clear all updates buffered in this batch.
    void Clear();
    // The size of the database changes caused by this batch.
    //
    // This number is tied to implementation details, and may change across
    // releases. It is intended for LevelDB usage metrics.
    size_t ApproximateSize() const;
    // Copies the operations in "source" to this batch.
    //
    // This runs in O(source size) time. However, the constant factor is better
    // than calling Iterate() over the source batch with a Handler that replicates
    // the operations into this batch.
    void Append(const WriteBatch& source);
    // Support for iterating over the contents of a batch.
    Status Iterate(Handler* handler) const;
private:
    friend class WriteBatchInternal;
    std::string rep_; // See comment in write_batch.cc for the format of rep_
};
```

WriteBatchInternal的定义

(db/write_batch_internal.h)

全部是静态函数,
并附带至少一个
WriteBatch*
batch参数

WriteBatchInternal
类是WriteBatch类
的友元类, 所以可
以访问rep_

```
class WriteBatchInternal {
public:
    // Return the number of entries in the batch.
    static int Count(const WriteBatch* batch);
    // Set the count for the number of entries in the batch.
    static void SetCount(WriteBatch* batch, int n);
    // Return the sequence number for the start of this batch.
    static SequenceNumber Sequence(const WriteBatch* batch);
    // Store the specified number as the sequence number for the start of
    // this batch.
    static void SetSequence(WriteBatch* batch, SequenceNumber seq);
    static Slice Contents(const WriteBatch* batch) { return Slice(batch->rep_); }
    static size_t ByteSize(const WriteBatch* batch) { return batch->rep_.size(); }
    static void SetContents(WriteBatch* batch, const Slice& contents);
    static Status InsertInto(const WriteBatch* batch, MemTable* memtable);
    static void Append(WriteBatch* dst, const WriteBatch* src);
};
```

我们对WriteBatch的修改及追加信息, 本质上是对rep_这个成员的修改。

rep_

WriteBatch那里有注释：

```
// WriteBatch::rep_ :=  
//     sequence: fixed64  
//     count: fixed32  
//     data: record[count]
```

1. 最开始8个字节用于存储Sequence，表示当前Writebatch中第一个操作所对应的序列号
2. 接下来4个字节用户存储Count，表示当前WriteBatch有多少个Operation
3. 紧接着就是count条Operation记录，每一个operation记录就是一个Put操作或者Delete操作

WriteBatch和WriteBatchInternal的实现

(db/write_batch.cc)

这几个函数其实是和数据的格式对应的。

EncodeFixed和DecodeFixed系列函数实现了数值到字符串的编解码。这些函数的实现竟然是在util/coding.h而不是util/coding.cc，有点不规范

```
// WriteBatch header has an 8-byte sequence number followed by a 4-byte count.
static const size_t kHeader = 12;
WriteBatch::WriteBatch() { Clear(); }
WriteBatch::~~WriteBatch() = default;
WriteBatch::Handler::~~Handler() = default;
void WriteBatch::Clear() {
    rep_.clear();
    rep_.resize(kHeader);
}
size_t WriteBatch::ApproximateSize() const { return rep_.size(); }
int WriteBatchInternal::Count(const WriteBatch* b) {
    return DecodeFixed32(b->rep_.data() + 8);
}
void WriteBatchInternal::SetCount(WriteBatch* b, int n) {
    EncodeFixed32(&b->rep_[8], n);
}
SequenceNumber WriteBatchInternal::Sequence(const WriteBatch* b) {
    return SequenceNumber(DecodeFixed64(b->rep_.data()));
}
void WriteBatchInternal::SetSequence(WriteBatch* b, SequenceNumber seq) {
    EncodeFixed64(&b->rep_[0], seq);
}
```

WriteBatch和WriteBatchInternal的实现 (db/write_batch.cc)

- Put和Delete首先将计数加一，在 rep_ 中写入操作类型，再写入键值对
- PutLengthPrefixedSlice会先写入字符串的长度，再写入字符串的内容。
- WriteBatchInternal的赋值和追加均是对 rep_ 的进行操作

```
void WriteBatch::Put(const Slice& key, const Slice& value) {
    WriteBatchInternal::SetCount(this, WriteBatchInternal::Count(this) + 1);
    rep_.push_back(static_cast<char>(kTypeValue));
    PutLengthPrefixedSlice(&rep_, key);
    PutLengthPrefixedSlice(&rep_, value);
}

void WriteBatch::Delete(const Slice& key) {
    WriteBatchInternal::SetCount(this, WriteBatchInternal::Count(this) + 1);
    rep_.push_back(static_cast<char>(kTypeDeletion));
    PutLengthPrefixedSlice(&rep_, key);
}

void WriteBatch::Append(const WriteBatch& source) {
    WriteBatchInternal::Append(this, &source);
}

void WriteBatchInternal::SetContents(WriteBatch* b, const Slice& contents) {
    assert(contents.size() >= kHeader);
    b->rep_.assign(contents.data(), contents.size());
}

void WriteBatchInternal::Append(WriteBatch* dst, const WriteBatch* src) {
    SetCount(dst, Count(dst) + Count(src));
    assert(src->rep_.size() >= kHeader);
    dst->rep_.append(src->rep_.data() + kHeader, src->rep_.size() - kHeader);
}
```

WriteBatch和WriteBatchInternal的实现 (db/write_batch.cc)

迭代函数和Handle部分
也很重要

WriteBatch::Iterate会按照顺序将 rep_ 中存储的键值对操作放到 handler 上执行。

```
Status WriteBatch::Iterate(Handler* handler) const {
    Slice input(rep_);
    if (input.size() < kHeader) {return Status::Corruption("malformed WriteBatch (too small)");}
    input.remove_prefix(kHeader);
    Slice key, value;
    int found = 0;
    while (!input.empty()) {
        found++;
        char tag = input[0];
        input.remove_prefix(1);
        switch (tag) {
            case kTypeValue:
                if (GetLengthPrefixedSlice(&input, &key) && GetLengthPrefixedSlice(&input, &value)) {
                    handler->Put(key, value);
                } else {return Status::Corruption("bad WriteBatch Put");}
                break;
            case kTypeDeletion:
                if (GetLengthPrefixedSlice(&input, &key)) {handler->Delete(key);}
                else {return Status::Corruption("bad WriteBatch Delete");}
                break;
            default:
                return Status::Corruption("unknown WriteBatch tag");
        }
    }
    if (found != WriteBatchInternal::Count(this)) {
        return Status::Corruption("WriteBatch has wrong count");
    } else {
        return Status::OK();
    }
}
```

WriteBatch和WriteBatchInternal的实现 (db/write_batch.cc)

Handle部分这里，下面的匿名空间里定义了一个继承 Handler的子类 MemTableInserter，将 Put和 Delete转到 MemTable上执行。

我们能看到Put和Delete都调用了mem_的Add函数(这是一个小坑)

```
namespace {  
class MemTableInserter : public WriteBatch::Handler {  
public:  
    SequenceNumber sequence_;  
    MemTable* mem_;  
  
    void Put(const Slice& key, const Slice& value) override {  
        mem_>Add(sequence_, kTypeValue, key, value);  
        sequence_++;  
    }  
    void Delete(const Slice& key) override {  
        mem_>Add(sequence_, kTypeDeletion, key, Slice());  
        sequence_++;  
    }  
};  
} // namespace
```

WriteBatchInternal::InsertInto

(db/write_batch.cc)

- 介绍了这么多相关的函数，我们才能到WriteBatchInternal::InsertInto（好不容易）
- 它的实质是直接根据 MemTable构造 MemTableInserter。

这样做有什么好处呢？

WriteBatch::Iterate与MemTable解耦，Handler可以自行替换。

综合来看，WriteBatch将所有的修改和删除操作均存储到一个字符串中，并且提供了内存数据库的迭代接口。

虽然这不是我们今天讲解的重点，但是今天透过WriteBatchInternal::InsertInto这个函数，相信大家对LevelDB的写入有了更深刻的认识。

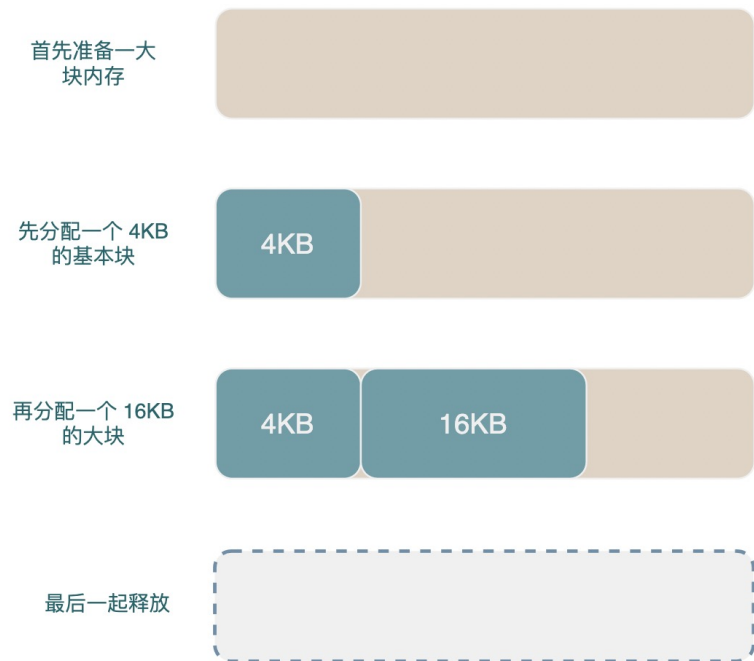
更进一步：
MemTable::Add
(db/memtable.cc)
Put和Delete都调用
了mem_的Add函
数

Add使用Arena进
行内存分配
经过格式转换之后
最终将数据插入了
跳表

```
void MemTable::Add(SequenceNumber s, ValueType type, const Slice& key,
                   const Slice& value) {
    // Format of an entry is concatenation of:
    //   key_size      : varint32 of internal_key.size()
    //   key bytes     : char[internal_key.size()]
    //   tag           : uint64((sequence << 8) | type)
    //   value_size    : varint32 of value.size()
    //   value bytes   : char[value.size()]
    // 键值对的格式为:
    // key_size: varint32 编码的 internal_key 长度
    // internal_key 内容
    // tag: 包含 SequenceNumber 和 type: uint64((sequence << 8) | type)
    // value_size: varint32 编码的 value 长度
    // value 内容
    size_t key_size = key.size();
    size_t val_size = value.size();
    // internal_key 由 user_key + uint64((sequence << 8) | type) 组成, 比 user_key 多一个 uint64 即 8 个字节
    size_t internal_key_size = key_size + 8;
    const size_t encoded_len = VarintLength(internal_key_size) +
                               internal_key_size + VarintLength(val_size) +
                               val_size; // 计算 entry 编码后大小
    char* buf = arena_.Allocate(encoded_len);
    char* p = EncodeVarint32(buf, internal_key_size);
    std::memcpy(p, key.data(), key_size);
    p += key_size;
    EncodeFixed64(p, (s << 8) | type); // 计算 tag, 存储到 buffer
    p += 8;
    p = EncodeVarint32(p, val_size);
    std::memcpy(p, value.data(), val_size);
    assert(p + val_size == buf + encoded_len);
    table_.Insert(buf); // 将键值对插入跳表中
}
```

Arena简要介绍 (util/arena.h, util/arena.cc)

- 是一种简单的内存分配器，Arena 会预先分配一大块内存，每次申请对象都从其中连续进行分配。Arena 中分配的对象不能单独释放，只能和 Arena 一起释放。



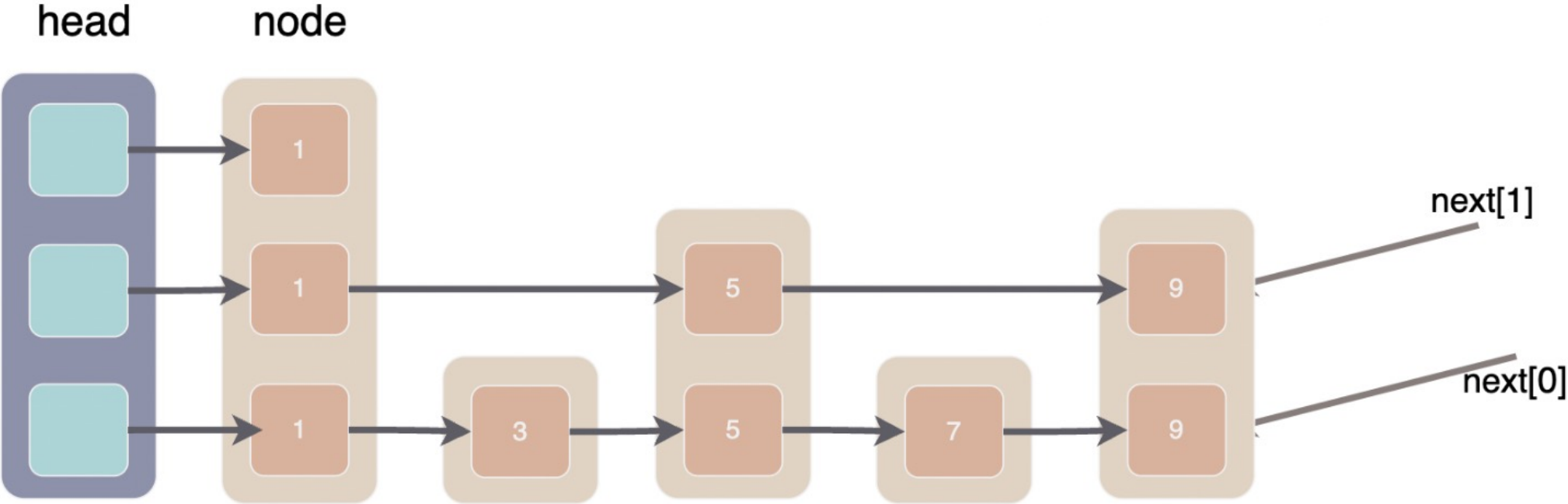
- `char* alloc_ptr_` 当前block里，下一次分配内存返回的地址
- `size_t alloc_bytes_remaining_` 当前block里剩余可分配的bytes

优点：显著的加快内存分配速度并减少内存碎片的产生；小内存分配效率高；方便统计内存使用情况

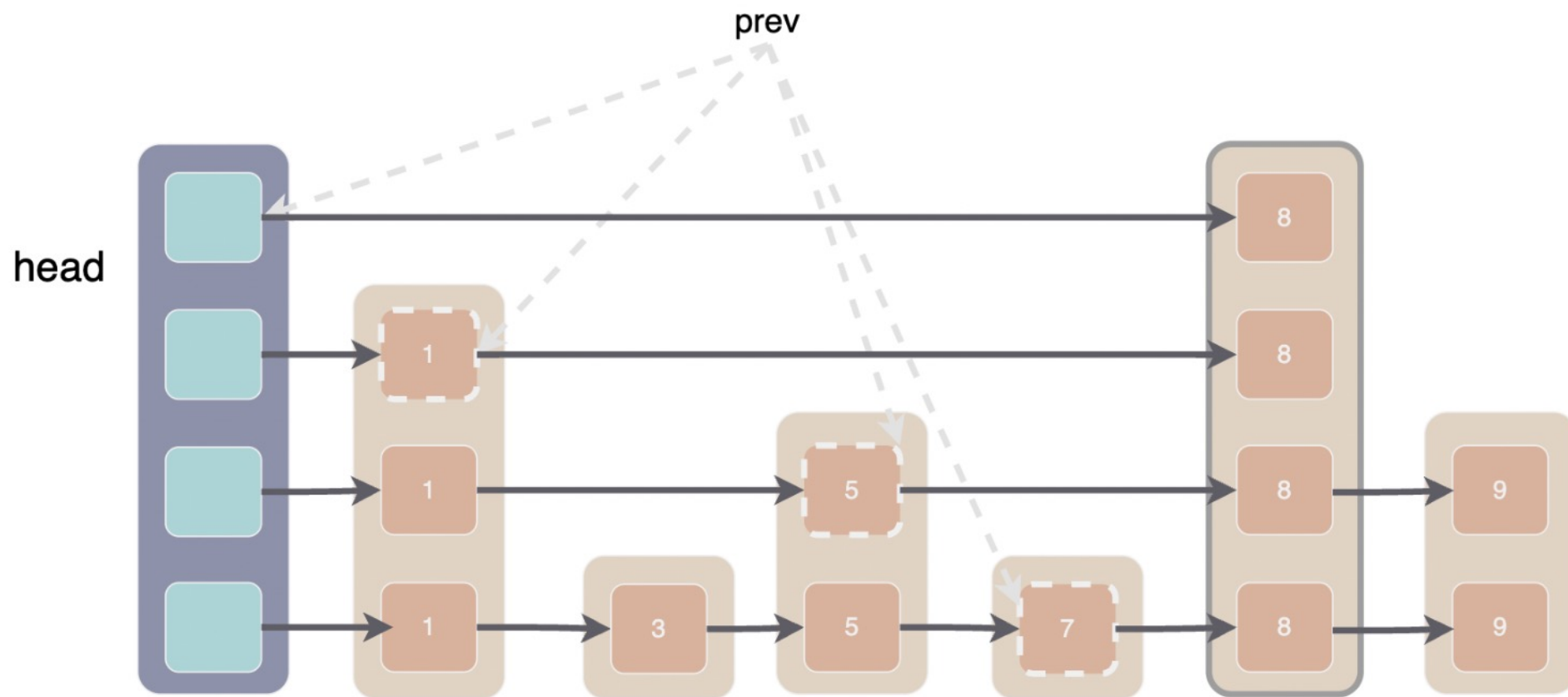
生命周期：Arena 的生命周期和memTable相同

刚刚我们也看到了，Add最后将数据插入跳表，insert的本质是跳表的插入，下面请另外一位同学来详细介绍它

Skiplist的基本结构



Write过程



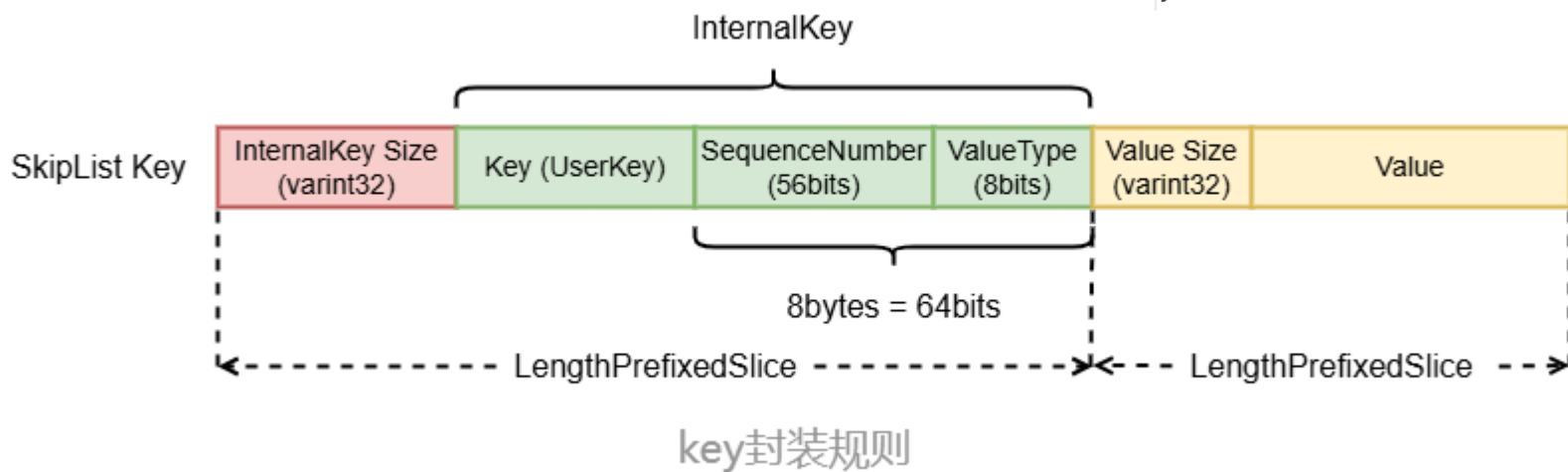
Skiplist主要代码路径

- Db/skiplist.h:Skiplist结构, 核心代码接口
- Util/arena.cc:为结点分配空间
- Db/dbformat.cc:提供skiplist结点比较范式

skiplist数据库内的应用

- 从具体数据来看

```
int InternalKeyComparator::Compare(const Slice& akey, const Slice& bkey) const {  
    // Order by:  
    //     increasing user key (according to user-supplied comparator)  
    //     decreasing sequence number  
    //     decreasing type (though sequence# should be enough to disambiguate)  
    int r = user_comparator_ -> Compare(ExtractUserKey(akey), ExtractUserKey(bkey));  
    if (r == 0) {  
        const uint64_t anum = DecodeFixed64(akey.data() + akey.size() - 8);  
        const uint64_t bnum = DecodeFixed64(bkey.data() + bkey.size() - 8);  
        if (anum > bnum) {  
            r = -1;  
        } else if (anum < bnum) {  
            r = +1;  
        }  
    }  
    return r;  
}
```



• 从写功能来看

没有delete功能，因为delete以日志方式添加，本质还是add

```
template <typename Key, class Comparator>
- class SkipList {
    private:
-     struct Node;

    public:
        // Create a new SkipList object that will use "cmp" for comparing keys,
        // and will allocate memory using "*arena". Objects allocated in the arena
        // must remain allocated for the lifetime of the skiplist object.
-     explicit SkipList(Comparator cmp, Arena* arena);

        SkipList(const SkipList&) = delete;
        SkipList& operator=(const SkipList&) = delete;

        // Insert key into the list.
        // REQUIRES: nothing that compares equal to key is currently in the list.
-     void Insert(const Key& key);

        // Returns true iff an entry that compares equal to key is in the list.
-     bool Contains(const Key& key) const;

        // Iteration over the contents of a skip list
>     class Iterator {...};
```

核心代码接口

在这里对于一些原子性操作会有些疑问，我们在后续的并发问题里会好好介绍。

这里值得注意的点，RandomHeight并不是我们平时所用的一些分布，例如(均匀分布，正态分布)，以及kMaxHeight为什么要设置成12。

```
namespace levelup {
void SkipList<Key, Comparator>::Insert(const Key& key) {
    // here since Insert() is externally synchronized.
    Node* prev[kMaxHeight];
    Node* x = FindGreaterOrEqual(key, prev);

    // Our data structure does not allow duplicate insertion
    assert(x == nullptr || !Equal(key, x->key));

    int height = RandomHeight();
    if (height > GetMaxHeight()) {
        for (int i = GetMaxHeight(); i < height; i++) {
            prev[i] = head_;
        }
        // It is ok to mutate max_height_ without any synchronization
        // with concurrent readers. A concurrent reader that observes
        // the new value of max_height_ will see either the old value of
        // new level pointers from head_ (nullptr), or a new value set in
        // the loop below. In the former case the reader will
        // immediately drop to the next level since nullptr sorts after all
        // keys. In the latter case the reader will use the new node.
        max_height_.store(height, std::memory_order_relaxed);
    }

    x = NewNode(key, height);
    for (int i = 0; i < height; i++) {
        // NoBarrier_SetNext() suffices since we will add a barrier when
        // we publish a pointer to "x" in prev[i].
        x->NoBarrier_SetNext(i, prev[i]->NoBarrier_Next(i));
        prev[i]->SetNext(i, x);
    }
}
```

RandomHeight

确定新节点层数的过程：

- 开始于最低层：新节点至少会出现在第一层, 也就是从第一层开始抛硬币.
- 抛硬币：对于每一层，你可以想象自己在做一个“抛硬币”的实验，如果硬币正面朝上（这个事件发生的概率为 p ），新节点的层数就会+1.
- 重复抛硬币直到失败：重复这个随机过程直到硬币背面朝上为止（即事件发生的概率为 $1 - p$ ），此时抛硬币过程结束, 新节点层数计算完成.
- 限制最大层数：SkipList通常有一个预设的最大层数 `MaxLevel`，节点的层数不会超过这个值.若抛硬币一直抛到第 `MaxLevel` 层，还是正面朝上，也不会再继续增加层数了

```
template <typename Key, class Comparator>
int SkipList<Key, Comparator>::RandomHeight() {
    // Increase height with probability 1 in kBranching
    static const unsigned int kBranching = 4;
    // 从1开始抛硬币
    int height = 1;
    // (rnd_.Next() % kBranching) == 0
    // 这个条件限制了上层的节点数量为下层节点数量的 1/4
    while (height < kMaxHeight && rnd_.OneIn(kBranching)) {
        // rnd_.Next() 生成一个随机数,
        // rnd_.Next() % 4 的意思是, 生成一个0~3的随机数,
        // 0,1,2,3 的概率都是1/4.
        // 所以(rnd_.Next() % 4) == 0 成立的概率是1/4.
        // 也就是说每次抛硬币都有1/4的概率层高+1.
        // 所以LevelDB的SkipList里, 概率因子是1/4.
        height++;
    }
    assert(height > 0);
    assert(height <= kMaxHeight);
    return height;
}
```

新问题：Skiplist为什么要设置成12（魔数）

推导思路：

在跳表中，每个节点出现在第 l 层的概率是 p^l ，因此第 l 层的节点数 m_l 近似为第 0 层节点数 n 的一部分，满足：

$$m_l \approx n \cdot p^l$$

而跳表的性能依赖于平均搜索路径的长度。如果最大高度是 h ，则最顶层（第 $h-1$ 层）的节点数大约为 1 个，因此我们有以下公式：

$$m_{h-1} \approx n \cdot p^{h-1} \approx 1$$

从这个公式可以推导出 n 的估算公式：

$$n \cdot p^{h-1} \approx 1$$

$$n \approx \frac{1}{p^{h-1}}$$

因此，跳表的节点总数 n 与最大高度 h 和概率因子 p 的关系为：

$$n \approx \frac{1}{p^{h-1}} = p^{-(h-1)}$$

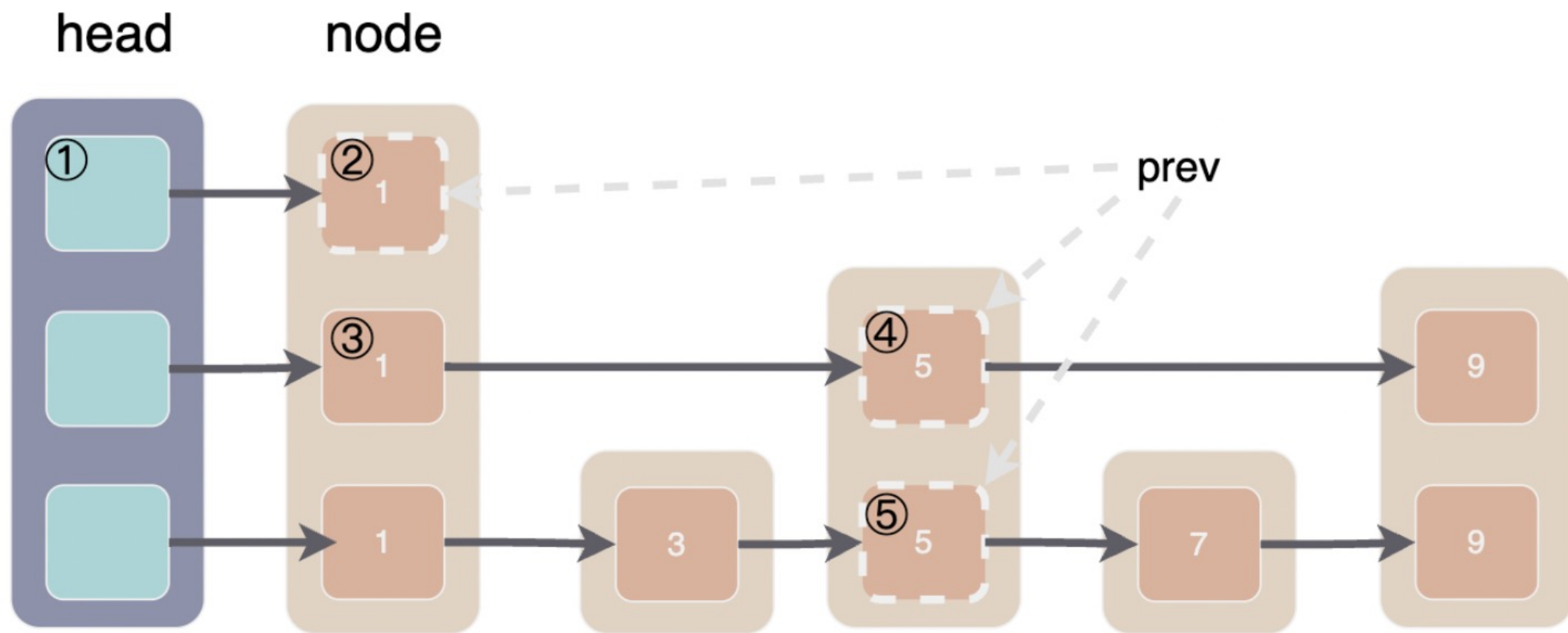
带入数据后，发现则最大可容的节点数 $n=4,194,304$ 。

也就是说，高度为12的SkipList最大可容纳400万条记录，只有超过400万规模，才需要增加高度以保证性能不下降。

当SkipList的数据规模在400万以内时，增加高度只会增加空间开销，而不会提升性能。

.

Skiplist的读取数据



- 读取数据的上层调用

```
// Unlock while reading from files and memtables
{
    mutex_.Unlock();
    // First look in the memtable, then in the immutable memtable (if any).
    LookupKey lkey(key, snapshot);
    if (mem->Get(lkey, value, &s)) {
        // Done
    } else if (imm != nullptr && imm->Get(lkey, value, &s)) {
        // Done
    } else {
        s = current->Get(options, lkey, value, &stats);
        have_stat_update = true;
    }
    mutex_.Lock();
}
```

```
leveldb::SkipList::FindGreaterOrEqual skiplist.h:271
leveldb::SkipList::Iterator::Seek skiplist.h:223
leveldb::MemTable::Get memtable.cc:105
leveldb::DBImpl::Get db_impl.cc:1153
GetData db_test1.cc:56
main db_test1.cc:80
```

当同一个键出现在多个MemTable 中时，应该返回最新版本的数据。
一个MemTable出现多个Key时，返回最新的key。

具体代码

```
typename SkipList<Key, Comparator>::Node*
SkipList<Key, Comparator>::FindGreaterOrEqual(const Key& key,
                                              Node** prev) const {
    Node* x = head_;
    int level = GetMaxHeight() - 1;
    while (true) {
        // 获取当前level层的下一个节点
        Node* next = x->Next(level);

        // KeyIsAfterNode实际上就是使用 Comparator 比较 Key 和 next->key
        // key > next->key: return true
        // key <= next->key OR key == nullptr: return false

        if (KeyIsAfterNode(key, next)) {
            // Keep searching in this list
            x = next;
        } else {
            // 当前待找节点比next-key小,
            // 需要往下一层查找.

            // prev 数组主要记录的就是每一层的 prev
            // 节点, 主要用于插入和删除时使用
            if (prev != nullptr) prev[level] = x;
            if (level == 0) {
                return next;
            } else {
                // Switch to next list
                level--;
            }
        }
    }
}
```

```
// Accessors/mutators for links.  Wrapped in methods so we can
// add the appropriate barriers as necessary.
Node* Next(int n) {
    assert(n >= 0);
    // Use an 'acquire load' so that we observe a fully initialized
    // version of the returned Node.
    return next_[n].load(std::memory_order_acquire);
}
```

```
template <typename Key, class Comparator>
bool SkipList<Key, Comparator>::KeyIsAfterNode(const Key& key, Node* n) const {
    // null n is considered infinite
    return (n != nullptr) && (compare_(n->key, key) < 0);
}
```

```

namespace leveldb {
Status DBImpl::Get(const ReadOptions& options, const Slice& key,

// Unlock while reading from files and memtables
{
    mutex_.Unlock();
    // First look in the memtable, then in the immutable memtable (if any)
    LookupKey lkey(key, snapshot);
    if (mem->Get(lkey, value, &s)) {
        // Done
    } else if (imm != nullptr && imm->Get(lkey, value, &s)) {
        // Done
    } else {
        s = current->Get(options, lkey, value, &stats);
        have_stat_update = true;
    }
    mutex_.Lock();
}
}

```

```

namespace leveldb {
Status DBImpl::Write(const WriteOptions& options, WriteBatch* updates) {
    // into mem_.
    {
        //memTable是线程安全的
        //unlock允许其他write放入writers队列中
        mutex_.Unlock();
        status = log_>AddRecord(WriteBatchInternal::Contents(write_batch));
        bool sync_error = false;
        if (status.ok() && options.sync) {
            status = logfile_>Sync();
            if (!status.ok()) {
                sync_error = true;
            }
        }
        if (status.ok()) {
            status = WriteBatchInternal::InsertInto(write_batch, mem_);
        }
        mutex_.Lock(); //后续将对writers队列操作
    }
}
}

```

新疑问：当前的memTable读写线程都没有大锁保护，那它是如何保证数据的正确的处理的。

数据库内不能并发写入以及可以并发读取都很好理解，那么为什么可以在写入时读取呢。

Add 函数写入由两处：一是修改 `max_height`；二是向 `prev` 后面插入新节点

一是MaxHeight的修改

```
int height = RandomHeight();
if (height > GetMaxHeight()) {
    for (int i = GetMaxHeight(); i < height; i++) {
        prev[i] = head_;
    }
    // It is ok to mutate max_height_ without any synchronization
    // with concurrent readers. A concurrent reader that observes
    // the new value of max_height_ will see either the old value of
    // new level pointers from head_ (nullptr), or a new value set in
    // the loop below. In the former case the reader will
    // immediately drop to the next level since nullptr sorts after all
    // keys. In the latter case the reader will use the new node.
    max_height_.store(height, std::memory_order_relaxed);
}
```

```
inline int GetMaxHeight() const {
    return max_height_.load(std::memory_order_relaxed);
}
```

```
namespace leveldb {
SkipList<Key, Comparator>::FindGreaterOrEqual(const Key& key,
                                                Node** prev) const {
    Node* x = head_;
    int level = GetMaxHeight() - 1;
    while (true) {
        // 获取当前level层的下一个节点
        Node* next = x->Next(level);

        // KeyIsAfterNode实际上就是使用 Comparator 比较 Key 和 next->key
        // key > next->key: return true
        // key <= next->key OR key == nullptr: return false

        if (KeyIsAfterNode(key, next)) {
            // Keep searching in this list
            x = next;
        } else {
            // 当前待查找节点比next-key小,
            // 需要往下一层查找.

            // prev 数组主要记录的就是每一层的 prev
            // 节点, 主要用于插入和删除时使用
            if (prev != nullptr) prev[level] = x;
            if (level == 0) {
                return next;
            } else {
                // Switch to next list
                level--;
            }
        }
    }
}
```

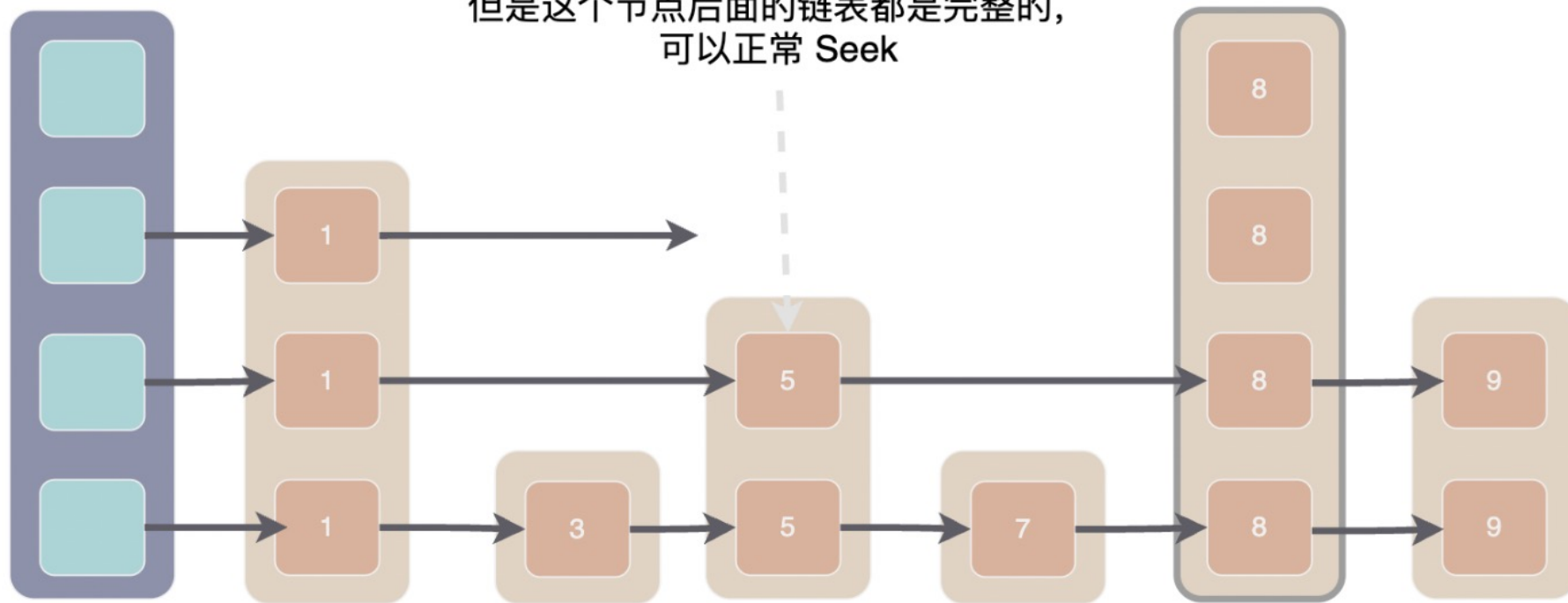

二是向 prev 后面插入新节点

```
x = NewNode(key, height);
for (int i = 0; i < height; i++) {
    // NoBarrier_SetNext() suffices since we will add a barrier when
    // we publish a pointer to "x" in prev[i].
    x->NoBarrier_SetNext(i, prev[i]->NoBarrier_Next(i));
    prev[i]->SetNext(i, x);
}

void SetNext(int n, Node* x) {
    assert(n >= 0);
    // Use a 'release store' so that anybody who reads through this
    // pointer observes a fully initialized version of the inserted node.
    next_[n].store(x, std::memory_order_release);
}

// No-barrier variants that can be safely used in a few locations.
Node* NoBarrier_Next(int n) {
    assert(n >= 0);
    return next_[n].load(std::memory_order_relaxed);
}
```

虽然 level2 上面的节点还没有连接，
但是这个节点后面的链表都是完整的，
可以正常 Seek



线程安全问题

这里回答两个问题

1. 在冻结 MemTable 之后，是否可能有一些线程仍然持有旧的 LSM 状态并写入这些不可变的 MemTable？如何防止这种情况发生？
2. 当不可变的 MemTable 落盘后还有线程在读这个 MemTable 怎么办？

在冻结 MemTable 之后，是否可能有一些线程仍然持有旧的 LSM 状态并写入这些不可变的 MemTable？如何防止这种情况发生？

问题可能产生诱因：

多线程时，可能会有一些线程提早引用了memTable，但当线程继续运行时，还未意识到MemTable 的状态已经改变

解决办法

1.通过互斥锁（如`MutexLock`）来确保写入操作的原子性，防止多线程同时修改MemTable。前置知识所说，主要是锁住writers对列，保证一个时刻内，只有一个leader writer，来负责写。（write函数内）

2.当MemTable被标记为不可变时，必须等待当前所有的写操作完成，这通常通过写入队列的机制来保证。线程在提交新的写操作时，会检查当前可写的 MemTable 状态，并确保不会继续写入冻结的MemTable。（MakeRoomForWrite函数）

```

MutexLock l(&mutex_);
writers_.push_back(&w);
while (!w.done && &w != writers_.front()) {
    w.cv.Wait();
}
if (w.done) {
    return w.status;
}

// May temporarily unlock and wait.
Status status = MakeRoomForWrite(updates == nullptr);
uint64_t last_sequence = versions_>LastSequence();
Writer* last_writer = &w;
if (status.ok() && updates != nullptr) { // nullptr batch :
    WriteBatch* write_batch = BuildBatchGroup(&last_writer);

```

```

Status DBImpl::MakeRoomForWrite(bool force) {
    break;
} else if (allow_delay && versions_>NumLevelFiles(0) >=
            config::kL0_SlowdownWritesTrigger) {
    // We are getting close to hitting a hard limit on the number of
    // L0 files. Rather than delaying a single write by several
    // seconds when we hit the hard limit, start delaying each
    // individual write by 1ms to reduce latency variance. Also,
    // this delay hands over some CPU to the compaction thread in
    // case it is sharing the same core as the writer.
    mutex_.Unlock();
    env_>SleepForMicroseconds(1000);
    allow_delay = false; // Do not delay a single write more than once
    mutex_.Lock();
} else if (!force &&
            (mem_>ApproximateMemoryUsage() <= options_.write_buffer_size)) {
    // There is room in current memtable
    break;
} else if (imm_ != nullptr) {
    // We have filled up the current memtable, but the previous
    // one is still being compacted, so we wait.
    Log(options_.info_log, "Current memtable full; waiting...\n");
    background_work_finished_signal_.Wait();

```

当不可变的 MemTable 落盘后还有线程在读这个 MemTable 怎么办？

引用计数：不可变的 MemTable 通常会有一个引用计数。当某个线程正在读取该 MemTable 时，会增加引用计数。在 MemTable 被刷盘到 SSTable 后，系统不会立即删除不可变 MemTable，会在内存继续保留。等待所有的读取操作完成，引用计数降为零，之后才会删除该 MemTable。

```
namespace leveldb {  
  
void Version::Ref() { ++refs_; }  
  
void Version::Unref() {  
    assert(this != &vset_->dummy_versions_);  
    assert(refs_ >= 1);  
    --refs_;  
    if (refs_ == 0) {  
        delete this;  
    }  
}
```

Q&A